

Mediapipe Palm Tracking (No-PTZ)

1. Experiment Objective

MediaPipe Hands is used to detect 21 keypoints of the palm, and the car rotates to track so that the palm stays at the center of the frame. It supports Tracking mode (rotation only) and Follow mode (rotation + distance keeping).

2. Experiment Principle

1. Keyword Explanation

Keyword	Description
MediaPipe Hands	A lightweight hand keypoint detection model provided by Google that can detect the coordinates of 21 hand keypoints in real time
21 Hand Keypoints	The hand skeleton nodes defined by MediaPipe, including fingertips, knuckles, palm, etc., numbered 0–20
Palm Keypoint (Point 5)	The MCP joint at the base of the index finger (numbered 5), used as the reference center for palm tracking
Hand Skeleton Connections	Connects the 21 keypoints according to the hand topology to form a skeleton for visualization
PD Control	Proportional-derivative controller that computes the body angular velocity based on the palm center deviation and its rate of change
Distance Following	Controls the car to move forward/backward based on the size change of the palm bounding box
QoS	best_effort (image input + cmd_vel downlink), depth=1

2. Technical Architecture

```
/camera/image_raw (BEST_EFFORT)
└→ hand_tracker_node
    └─ Image processing: image conversion → flip → MediaPipe Hands.process() →
    extract 21 keypoints
        └─ Target selection: palm center (5th keypoint)
            └─ Body control timer (10Hz): pixel error → PD angular velocity + bounding
            box size distance following → /cmd_vel (BEST_EFFORT)
```

Requires a MediaPipe Python environment (ROS 2 and workspace environment).

3. Core Topic Quick Reference

Firmware Uplink

Topic	Message Type	QoS	Description
/camera/image_raw	sensor_msgs/Image	best_effort	ESP32 camera image

Firmware Downlink

Topic	Message Type	QoS	Field	Range	Direction
/cmd_vel	geometry_msgs/Twist	best_effort + keep_last(1)	linear.x	-0.20~0.20 m/s	$\pm$ forward, $\mp$ backward
			angular.z	-0.60~0.60 rad/s	$\pm$ turn left, $\mp$ turn right

3. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
PTZ version	✗
No-PTZ version	☑
Required peripheral	ESP32 camera (bracket-mounted)

2. Start the Agent + Camera + Joint Model (Terminal 1)

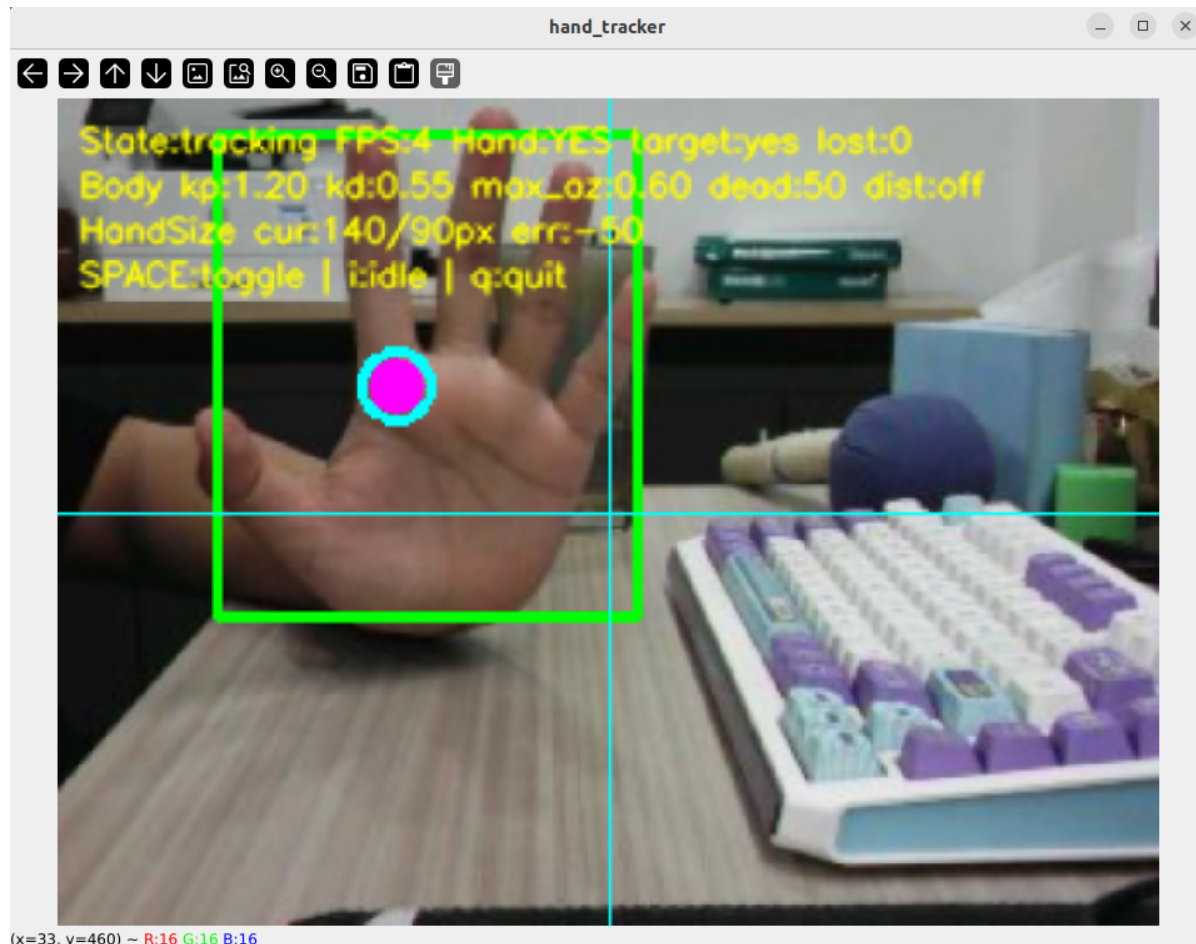
```
ros2 launch microros_v2_bringup car_base.launch.py
```

```
yahboom@yahboom:~$ ros2 launch microros_v2_bringup car_base.launch.py
[INFO] [launch]: All log files can be found below /home/yahboom/.ros/log/2026-07-28-15-04-46-269642-yahboom-579370
[INFO] [launch]: Default logging verbosity is set to INFO
[INFO] [micro_ros_agent-1]: process started with pid [579411]
[INFO] [camera_driver-2]: process started with pid [579413]
[INFO] [robot_state_publisher-3]: process started with pid [579415]
[INFO] [joint_state_publisher-4]: process started with pid [579417]
[INFO] [ekf_node-5]: process started with pid [579419]
[micro_ros_agent-1] [1785222287.404236] info | UDPv4AgentLinux.cpp | init | running... | port: 8090
[robot_state_publisher-3] [INFO] [1785222288.847816851] [robot_state_publisher]: got segment base_footprint
[robot_state_publisher-3] [INFO] [1785222288.848013042] [robot_state_publisher]: got segment base_link
[robot_state_publisher-3] [INFO] [1785222288.848028942] [robot_state_publisher]: got segment bwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848037122] [robot_state_publisher]: got segment cam1_Link
[robot_state_publisher-3] [INFO] [1785222288.848043874] [robot_state_publisher]: got segment cam2_Link
[robot_state_publisher-3] [INFO] [1785222288.848050538] [robot_state_publisher]: got segment cam3_Link
[robot_state_publisher-3] [INFO] [1785222288.848057264] [robot_state_publisher]: got segment imu_frame
[robot_state_publisher-3] [INFO] [1785222288.848063747] [robot_state_publisher]: got segment laser_frame
[robot_state_publisher-3] [INFO] [1785222288.848070240] [robot_state_publisher]: got segment lfwheel_Link
[robot_state_publisher-3] [INFO] [1785222288.848076888] [robot_state_publisher]: got segment rfwheel_Link
[joint_state_publisher-4] [INFO] [1785222289.372349515] [joint_state_publisher]: Waiting for robot_description to be published on the robot_description
[camera_driver-2] [INFO] [1785222289.716161193] [esp32_ip_camera_publisher]: camera node started, camera_url: http://192.168.12.37:81/stream
[camera_driver-2] [WARN] [1785222292.705741414] [esp32_ip_camera_publisher]: Camera not opened, trying reconnect... (next retry in 1.0s)
[camera_driver-2] [INFO] [1785222294.072701944] [esp32_ip_camera_publisher]: IP camera stream opened successfully
```

3. Start Palm Tracking (Terminal 2)

Tracking mode:

```
ros2 launch microros_v2_no_ptz_visual_control_pkg hand_tracker.launch.py
```



6. Operation Instructions

Key	Function
Space	Toggle tracking/standby state
i / I	Switch to the standby (IDLE) state
q / Q / ESC	Exit the program

4. Experiment Phenomenon

- When the palm moves in the frame, the car automatically rotates to track it, keeping the palm at the center of the frame
- The 21 hand keypoints and skeleton connections are displayed in real time
- Follow mode: the car backs away when the palm moves closer and moves forward when it moves away
- After the palm moves out of the frame, the car stops; when it reappears, tracking resumes automatically

5. Code Explanation

Source path:

- Launch (track):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/hand_tracker.launch.py`
- Launch (follow):
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/hand_follow.launch.py`
- Node source:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/hand_tracker_node.py`
- Hand detector:
`~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/hand_detector.py`

1. Core Node Analysis

`hand_tracker_node` uses `HandDetector` to wrap the MediaPipe Hands model. Core workflow:

- **Image processing** (15Hz): raw Image frame → `cv_bridge` conversion → flip → `HandDetector.findHands()` → extract 21 keypoints
- **Target selection**: the 5th keypoint (index finger MCP joint) is used as the palm center
- **Body control** (10Hz): deviation between the palm center x-coordinate and the image center → PD angular velocity; bounding box size deviation → PD linear velocity

Constructor — create the MediaPipe detector and publishers/subscribers:

```
# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visual_control_pkg/hand_tracker_node.py

def __init__(self) -> None:
    super().__init__(DEFAULT_NODE_NAME)
    self._tgt_lock = threading.Lock()
    self._state = STATE_IDLE
    self._has_target = False
    self._lost_frames = 0
    self._target_x: Optional[float] = None
    self._target_y: Optional[float] = None
    self._hand_size: Optional[float] = None # Palm bounding box size

    self._declare_params()
    self._load_params()

    # MediaPipe hand detector
    self._hand_det = HandDetector()

    best_effort = QoSProfile(
        history=QoSHistoryPolicy.KEEP_LAST, depth=1,
        reliability=QoSReliabilityPolicy.BEST_EFFORT,
        durability=QoSDurabilityPolicy.VOLATILE,
    )
    self._img_sub = self.create_subscription(
        Image, self._img_topic, self._on_image, best_effort)
    self._cmd_pub = self.create_publisher(Twist, self._cmd_topic, best_effort)
```

```

self._img_timer = self.create_timer(
    1.0 / max(float(self._img_rate), MINIMUM_TIMER_RATE_HZ),
self._process_image)
self._body_timer = self.create_timer(
    1.0 / max(float(self._body_rate), MINIMUM_TIMER_RATE_HZ),
self._body_tick)

```

Image processing callback — MediaPipe detection + keypoint drawing:

```

# Source file:
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz_visua
l_control_pkg/hand_tracker_node.py

def _process_image(self) -> None:
    """Image processing."""
    with self._img_lock:
        if self._latest_frame is None:
            return
        frame = self._latest_frame.copy()

    if frame is None:
        return
    frame = cv.resize(frame, (self._img_w, self._img_h))
    if self._flip_h:
        frame = cv.flip(frame, 1)

    # MediaPipe Hands detection + skeleton drawing
    frame, _ = self._hand_det.findHands(frame, draw=True)

    with self._tgt_lock:
        if len(self._hand_det.lmList) != 0:
            # Take the 5th keypoint (index finger MCP) as the palm center
            mx, my = self._hand_det.lmList[5][1], self._hand_det.lmList[5][2]
            # Compute the bounding box size (used for distance following)
            x_vals = [pt[1] for pt in self._hand_det.lmList]
            y_vals = [pt[2] for pt in self._hand_det.lmList]
            self._target_x = float(mx)
            self._target_y = float(my)
            self._hand_size = max(max(x_vals) - min(x_vals), max(y_vals) -
min(y_vals))
            self._has_target = True
            self._lost_frames = 0
        else:
            self._lost_frames += 1
            if self._lost_frames > self._lost_frame_threshold:
                self._has_target = False

```

2. Key Parameters

Parameter definition file:

- Launch:


```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/launch/hand_tracker.launch.py
```
- Node:


```
~/microros_ws/src/microros_v2_no_ptz_visual_control_pkg/microros_v2_no_ptz
```

Parameter	Type	Default	Description
<code>deadband_x_pixel</code>	float	<code>50.0</code> px	Horizontal deadband
<code>angular_kp</code>	float	<code>1.2</code>	Body angle PD proportional coefficient
<code>angular_kd</code>	float	<code>0.55</code>	Body angle PD derivative coefficient
<code>max_angular_z</code>	float	<code>0.6</code> rad/s	Maximum body steering speed
<code>lost_frame_threshold</code>	int	<code>5</code>	Consecutive lost-frame tolerance threshold
<code>enable_distance_follow</code>	bool	<code>false</code>	Whether to enable distance following
<code>target_hand_size_px</code>	float	<code>90.0</code> px	Target palm bounding box size for distance following
<code>distance_tolerance_hand_size_px</code>	float	<code>12.0</code> px	Distance-following deadband
<code>distance_kp</code>	float	<code>0.05</code>	Distance PD proportional coefficient
<code>max_linear_x</code>	float	<code>0.2</code> m/s	Maximum forward/backward speed
<code>image_processing_rate_hz</code>	float	<code>15.0</code> Hz	Image processing rate
<code>body_control_rate</code>	float	<code>10.0</code> Hz	Body control rate

3. Node Description

Node	Package	Description
<code>hand_tracker_node</code>	<code>microros_v2_no_ptz_visual_control_pkg</code>	MediaPipe palm tracking with 21 keypoints + PD body control

6. Common Problems

Problem	Cause	Solution
Sudden palm loss	Palm flipped or occluded	Keep the palm plane facing the camera
Palm is not detected	Distance too far or insufficient lighting	Move closer to the camera and improve ambient lighting
Tracking jitter	Deadband too small	Increase <code>deadband_x_pixel</code> appropriately